SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	B.NANDINI	**Reg. No.:**	192525137
Date:	23-08-2026	**Duration:**	60 minutes

Code of Conduct

I B . NANDINI (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.NANDINI

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.

   Frequent updates to equipment, booking, and resource modules.

   Automatic testing after every code change.

   Continuous integration of frontend, backend, and database code.

   Automated detection of bugs and security issues.

   Reliable deployment to staging and production.

   Quick rollback if deployment fails.

   Notifications about build, test, and deployment status.

2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.

Developer

↓

GitHub Repository

↓

Code Commit / Pull Request

↓

Build

↓

Automated Testing

↓

Code Quality & Security Analysis

↓

Package / Docker Image

↓

Staging Deployment

↓

Acceptance Testing

↓

Production Deployment

↓

Monitoring & Notifications

↓

Rollback if Failure

3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.

Stage	Tool	Purpose
Source Code	Git + GitHub	Version control
CI/CD	GitHub Actions	Automate pipeline
Build	npm / Maven	Build application
Testing	Jest / PyTest	Automated testing
API Testing	Postman/Newman	Test backend APIs
Code Quality	SonarQube	Detect code issues
Security	OWASP Dependency-Check	Find vulnerabilities
Packaging	Docker	Create deployable container
Deployment	Docker / Kubernetes	Deploy application
Monitoring	Prometheus	Monitor application
Notifications	Email/Slack	Pipeline alerts

4. Define appropriate **automated testing strategies** to be integrated into the pipeline.

   - **Unit Testing:** Test individual functions such as booking and rental-cost calculation.
   - **Integration Testing:** Check communication between frontend, backend, and database.
   - **API Testing:** Verify login, equipment search, booking, and resource APIs.
   - **Functional Testing:** Verify complete booking and rental workflows.
   - **Security Testing:** Check authentication and unauthorized access.
   - **Regression Testing:** Ensure new changes do not break existing features.
   - **Performance Testing:** Test the system with many users and equipment records.

5. Design the **deployment strategy**, including development, testing/staging, and production environments.

Development Environment

- Developers commit code to GitHub.
- Automatic build and unit tests are executed.
- Failed tests stop the pipeline.

Testing/Staging Environment

- Successful builds are deployed to staging.
- Integration, API, security, and acceptance tests are performed.
- Approval is required before production deployment.

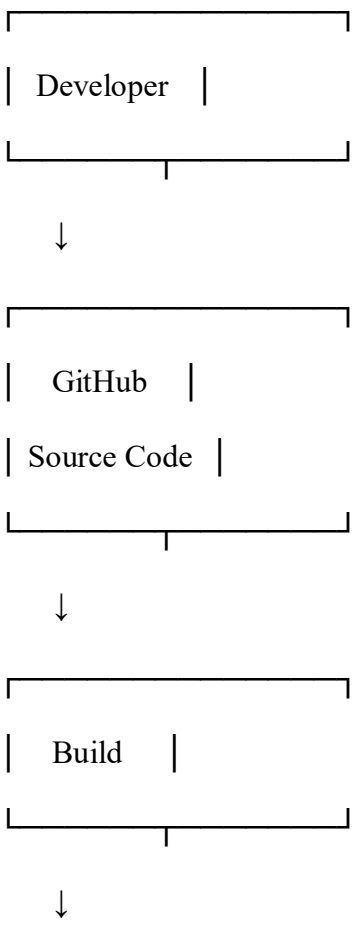
Production Environment

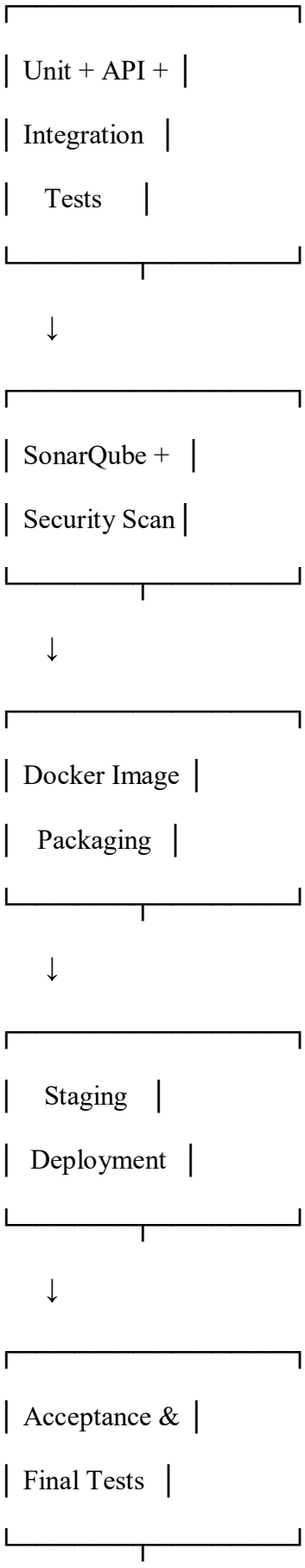
- Tested version is deployed.
- Application is monitored continuously.
- If a critical problem occurs, the previous stable version is restored.

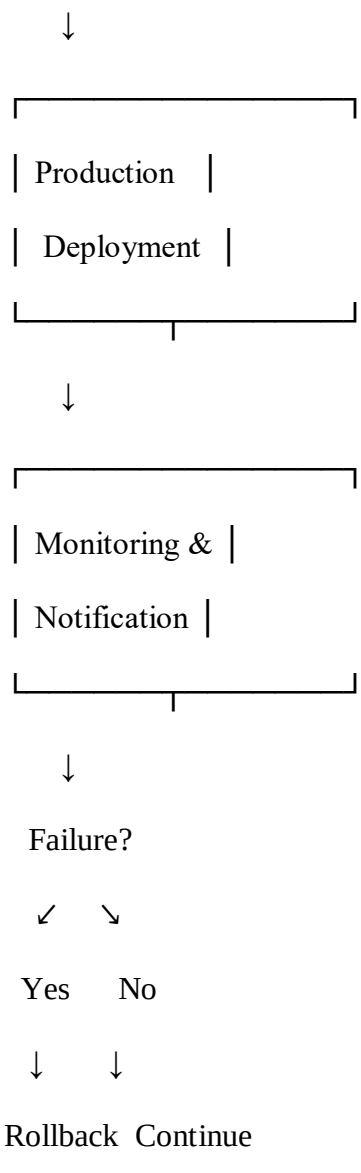
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.

    - Use GitHub secrets for passwords and API keys.
    - Perform dependency vulnerability scanning.
    - Use SonarQube for code-quality checks.
    - Require pull-request review before merging.
    - Run automated tests before deployment.
    - Send notifications for build/test/deployment results.
    - Maintain previous stable Docker image.
    - Automatically rollback when production health checks fail.

7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.




Unit + API + Integration Tests
SonarQube + Security Scan
Docker Image Packaging
Staging Deployment
Acceptance & Final Tests



Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollbac k details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/docum entation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4

Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25